FORESTER

Technical Report | HackApertus Track 2B

Track: Track 2B - Own Project

Event: Online

Team / members: Vicky Irwanto, Benny Fathurrahman, Sandy Rizki Fahmi, Faisal Alfa, Hafara Putri

Demo: https://youtu.be/BYLg8kDOW0U

1. Summary

Forester is a three-level forest defence game. Players place defences; Apertus chooses the enemy composition, routes and timing for finite waves. The game validates model plans before executing a deterministic offline simulation. It is a strategy prototype, not real-world wildfire training.

2. Architecture

A Unity WebGL client runs gameplay and presentation. A Python standard-library HTTP server in Docker serves the prebuilt player and a same-origin plan gateway. The gateway calls an OpenAI-compatible Apertus endpoint. Configuration is through LLM_NAME, LLM_BASE_URL and LLM_API_KEY; the upstream key is never returned to the browser. The gateway session token is ephemeral. Build time requires Unity 6000.6.2f1 with WebGL support; runtime requires Docker, a WebGL2 browser, and an Apertus endpoint for AI mode. No Unity installation is needed by judges.

Target architecture

On-premise deployable: run Docker and the browser on organisation-managed infrastructure and configure an organisation-managed HTTPS Apertus endpoint. The default CSCS demonstration has an external network dependency. A local model serving stack/weights are not bundled or tested; do not interpret this as a verified air-gapped or Swiss-cloud deployment. Vercel is an optional demo hosting path, not evidence of sovereign deployment.

3. Use of Apertus

Default model: swiss-ai/Apertus-v1.5-70B, hosted inference. The prompt, schema and gateway validation code are in src/ForesterGateway. The model receives a bounded structured game observation: board and legal routes, current defenders, resource/integrity state, previous outcomes, enemy catalogue, allowed conditions and wave bounds. The model returns a JSON threat plan. It does not control Unity objects directly. Invalid plans are rejected; game-side scheduling fits legal tactics to finite spawn windows. No fine-tuning is performed. The scripted mode is an explicitly labelled baseline, not Apertus inference.

4. Data

No external research dataset or personal gameplay dataset is supplied. data/ is empty. Source assets are shipped for reproducibility with their original licence constraints; the team must audit redistribution rights before public submission. User keys, local caches, generated logs and private configuration are excluded.

5. Evaluation

Web export built successfully with zero build errors. Local browser navigation covered menu, all three map markers, gameplay and 16:9 / 16:10 viewport sizing. Two browser-routing tests, four Node relay tests and five Python web-server tests passed during export preparation. These are engineering checks, not a measured AI win-rate benchmark. No statistically supported 50:50 balance claim is made. Docker make run built and started a healthy container on the development Mac. Health/config and player assets returned HTTP 200; 30 Python tests passed. Fresh live inference remains unverified; see docs/VERIFICATION.txt.

6. Limitations

A valid permitted Apertus key and reachable endpoint are required for AI mode. Browser CSCS calls need a CORS-compatible gateway. Model output can be invalid or slow; bounded validation/fallback protects gameplay. One URP FSR shader warning was observed in WebGL. Browser/device coverage and gameplay balance remain limited.

7. Reproducibility

From track_2b, set the three LLM environment variables and run make run. Open localhost:8080. No-key startup is scripted mode. src/Unity contains the editable Unity project and src/Web/player the prebuilt export. Record the final repository commit, actual Docker verification and evaluation hardware before submission.

8. Next steps

Measure valid-plan rate, inference latency and win rates across fixed strategy seeds; verify local Apertus hosting; improve browser performance and accessibility.

License

Report: CC-BY-4.0, following the report template. Third-party code/assets retain their respective licences. Final team licensing and asset audit are pending.

References

https://github.com/HackApertus/project-template/tree/main/track_2b

https://docs.cscs.ch/services/inference/api/